CSE 323: Operating System Design

File System

Salman Shamil



North South University (NSU)
Fall 2025

Original slides by Mathias Payer and Sanidhya Kashyap [EPFL]

Lecture Topics

- Filesystem API
- Internal and external interface
- Inodes and devices
- Filesystem implementation

This slide deck covers chapters 39, 40, 41 in OSTEP.

Purpose of a File System (1/2)

- Given: set of persistent blocks from a storage device
- Goal: manage these blocks efficiently.

Purpose of a File System (1/2)

- Given: set of persistent blocks from a storage device
- Goal: manage these blocks efficiently.

- Manages data (mostly) on nonvolatile storage
- Enables users to name and manipulate semi-permanent files
- Provide mechanisms to organize files and their metadata
  - e.g., owner, permissions, or type

Purpose of a File System (2/2)

- Map bytes on disk to "file"
- Share files (concurrently?) among users and processes
  - Decide on locking granularity and binding operations
  - Semantics of operations like truncating in a shared world
- File caching: metadata and contents

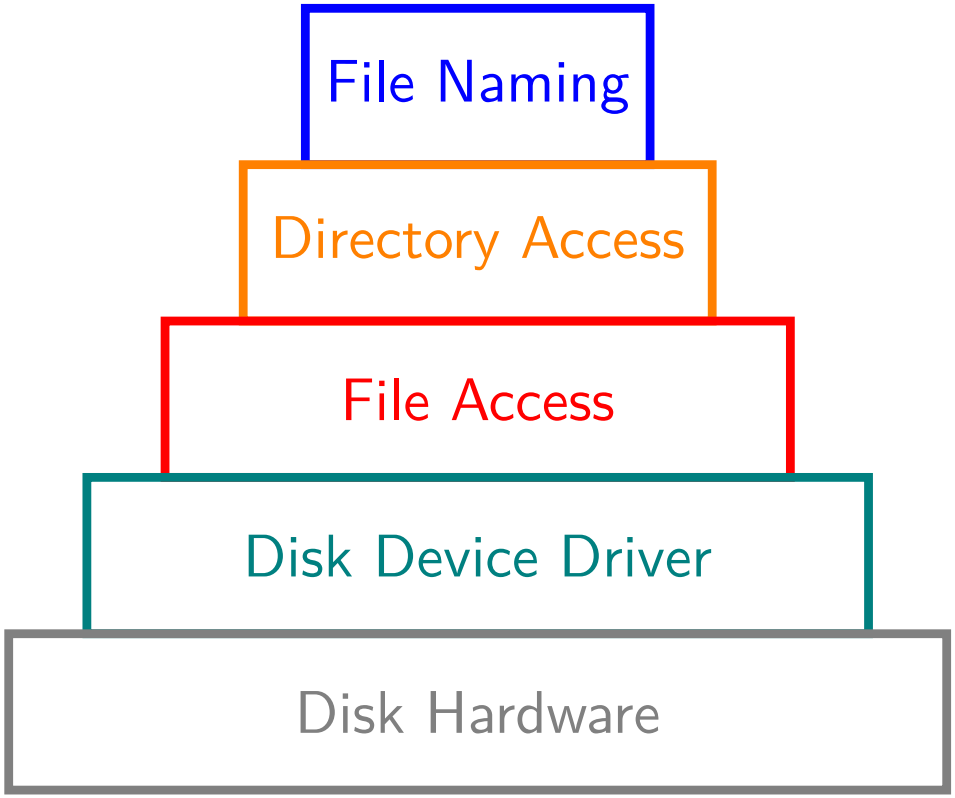
The `file` Abstraction

- A file is a linear persistent array of bytes
  - Data operations: read or write
  - Metadata operations: create, delete, modify permissions/user/...
- Directory contains subdirectories
  - List of directories, files, inode mappings

The `file` Abstraction

- Different perspectives
  - File name (human readable)
  - Inode and device number (persistent ID)
  - File descriptor (process view)

The I/O Hierarchy



- Each level adds functionality (and complexity)

Different Functionalities

- ***Naming***
  - specifies name syntax and encoding
  - e.g., a URL; may be aware if a file is local/remote
- ***Directory access***
  - map name to file object
  - resolve a string to an object
- ***File access***
  - concerns file operations
  - e.g., create/delete/read/write

Different File Philosophies

- Typed files: associate structure
  - System defines *all* possible file types (e.g., text document, source file, html file)
  - File type set at creation, file type specifies operations
- Untyped files: array of bytes
  - File is a sequence of bytes
  - System does neither understand nor care about contents
  - File operations apply to all files

Different File Philosophies

- Typed files: associate structure
  - System defines *all* possible file types (e.g., text document, source file, html file)
  - File type set at creation, file type specifies operations
- Untyped files: array of bytes
  - File is a sequence of bytes
  - System does neither understand nor care about contents
  - File operations apply to all files
- Modern systems settled on untyped files.
- Each file now has attributes or even extended attributes.
  - executable, append only, COW etc.
  - check manpage for `chattr`

Desired File Operations

- `create`: create a new file
- `unlink`: remove/destroy a file
- `open`: map a path to a file identifier
- `close`: close a file identifier
- `read`: read from the current file position
- `write`: write to the current file position
- `seek`: modify the file position
- `control`: various control operations such as changing permissions or user
- `createdir`: create a new directory
- `rmdir`: remove a directory
- `readdir`: return all files in a directory

Desired cost of file operations

- Sequential read/write is common
  - Design goal: `O(size of transfer)`
- Random access (seeking) is infrequent
  - Design goal: `O(log file length)`

Desired cost of file operations

- Sequential read/write is common
  - Design goal: `O(size of transfer)`
- Random access (seeking) is infrequent
  - Design goal: `O(log file length)`
- Constraints and observations
  - Many files are small
  - A few files are large
  - Most access is sequential, few accesses are at random positions
- Ideas for a clever data structure?

The 3 views of a file

- ***Operating system:*** Inode and device id
  - Ids are unique and great and unambiguous
- User: file name
- Process: File descriptor

Managing files: inodes

- An `inode` contains metadata of a file
- Each file has exactly one associated inode
- Each inode is unique on a filesystem (not globally!)
- Inodes are recycled after reuse

Managing files: inodes

- An `inode` contains metadata of a file
- Each file has exactly one associated inode
- Each inode is unique on a filesystem (not globally!)
- Inodes are recycled after reuse
- Note: multiple file names may map to the same inode
  - see "hard links"

Table of inodes (1/2)

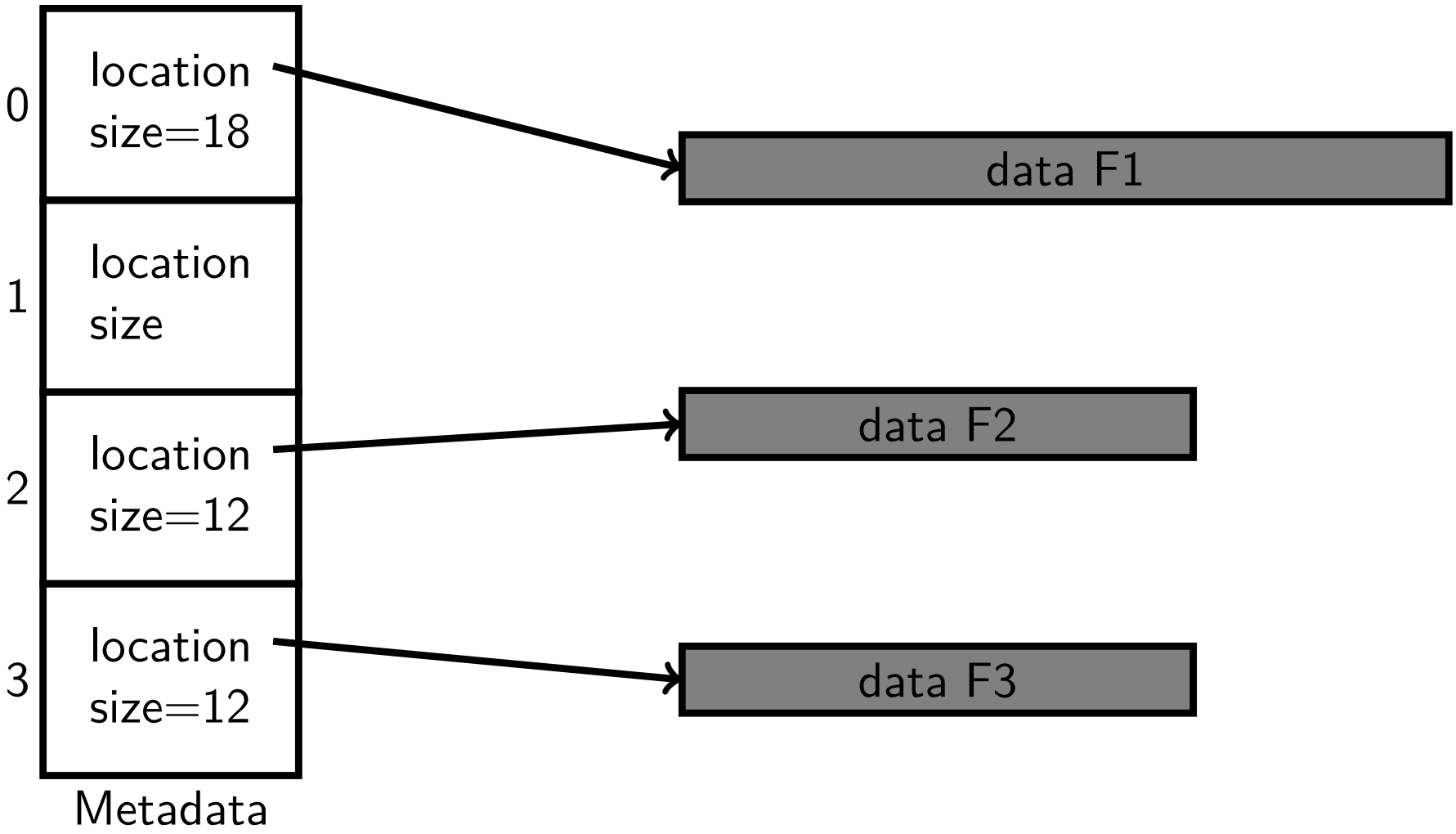


Table of inodes (2/2)

- Storage space is split into inode table and data storage
  - Initial prototypes used static sized table and data storage
- Files are statically allocated
- Need to remember inode number to access file content

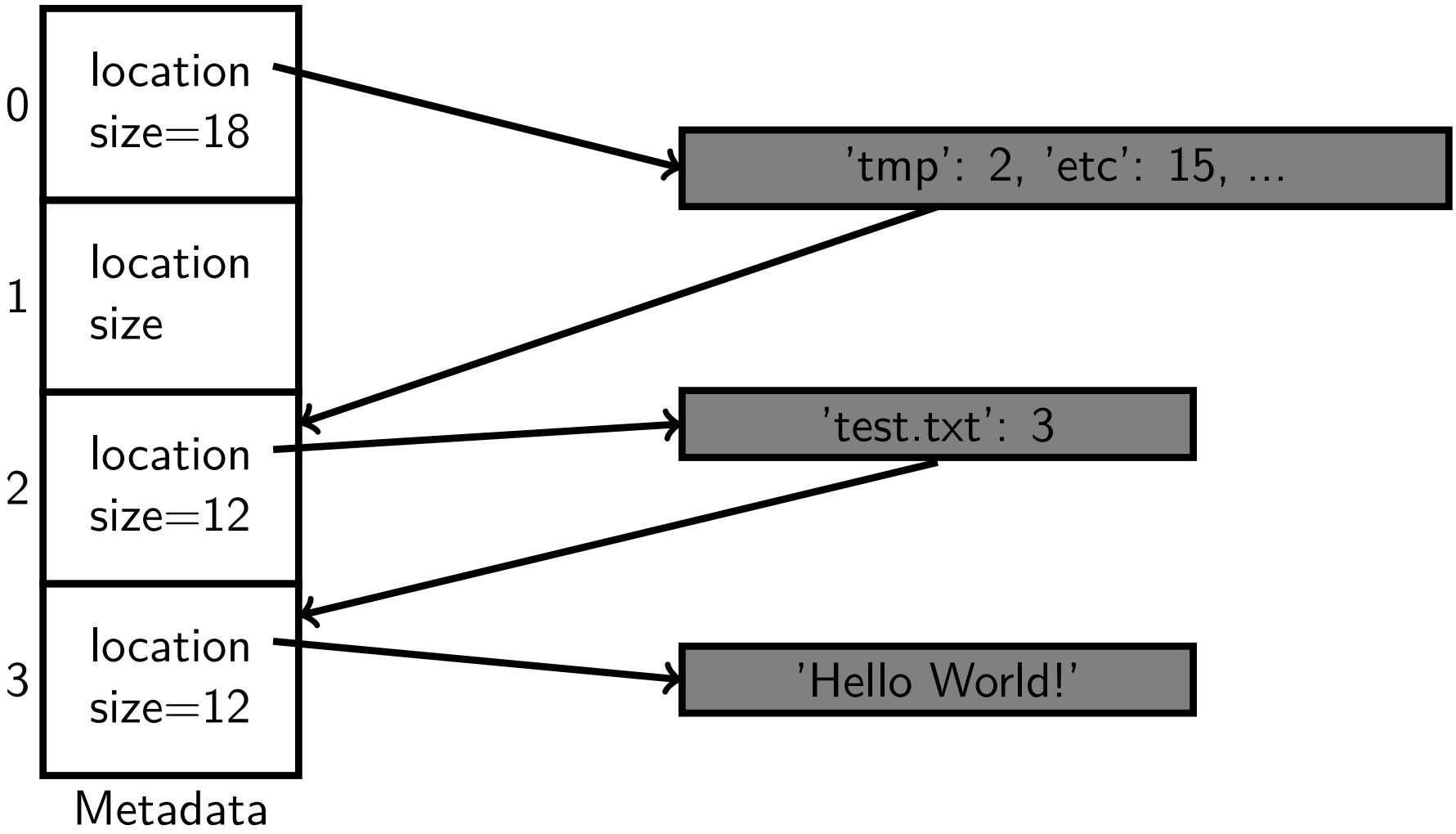
Table of inodes (2/2)

- Storage space is split into inode table and data storage
  - Initial prototypes used static sized table and data storage
- Files are statically allocated
- Need to remember inode number to access file content
- Idea: Use a dedicated place at the beginning of the storage medium, mostly initial blocks on the storage medium
  - A dedicated special file to store a mapping from file names to inodes

The 3 views of a file

- Operating system: Inode and device id
- ***User:*** file name
  - Humans are better at remembering names than numbers
- Process: File descriptor

From path to inode (1/2)



From path to inode (2/2)

- A special file stores mapping between file names and inodes
- Extend to hierarchy: mark if a file name maps to a regular file or a directory
  - Access to '/tmp/test.txt' in 3 steps: 'tmp', 'test.txt', contents
- What data should you store in the directory file (compared to the inode)?

Special directory entries: . and ..

```
$ ls -al
drwxr-xr-x 6 sanidhya sanidhya  4096 Nov 15 13:38 .
drwxr-xr-x 5 sanidhya sanidhya  4096 Nov  1 13:52 ..
-rw-r--r-- 1 sanidhya sanidhya  6830 Nov  1 13:52 00-intro.
-rw-r--r-- 1 sanidhya sanidhya 16726 Nov  1 13:52 11-proc.m
-rw-r--r-- 1 sanidhya sanidhya 19316 Nov  1 13:52 12-sched.
-rw-r--r-- 1 sanidhya sanidhya 14048 Nov  1 13:52 13-seg.mo
-rw-r--r-- 1 sanidhya sanidhya 17495 Nov  1 13:52 14-pg.md
-rw-r--r-- 1 sanidhya sanidhya  3603 Nov  1 13:52 15-virt.m
```

- '.'' maps to the current, '..'' maps to the next higher directory

The 3 views of a file

- Operating system: Inode and device id
- User: file name
- **Process:** File descriptor
  - Keeps track of per-process state (e.g., read position or name, inode mapping)

File descriptor (1/4)

- The combination of file names and inode/device id are sufficient to implement persistent storage

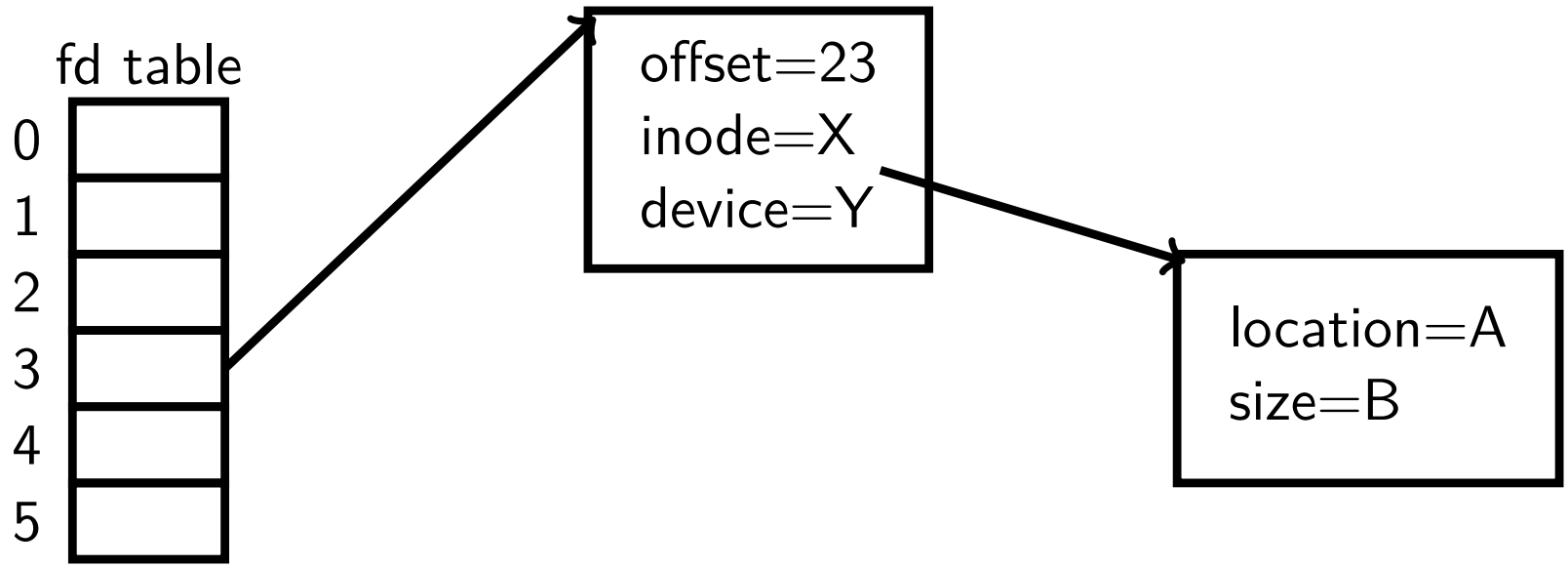
File descriptor (1/4)

- The combination of file names and inode/device id are sufficient to implement persistent storage
- Drawback: constant lookups from file name to inode/device id are costly

File descriptor (1/4)

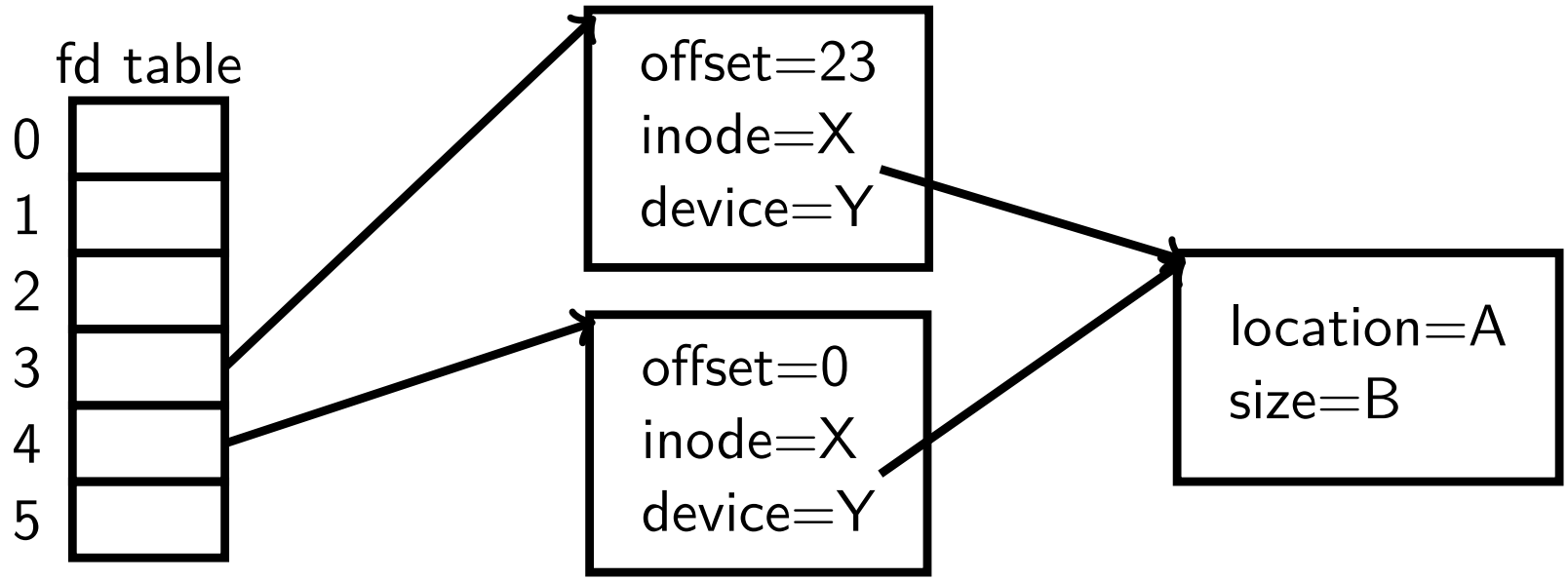
- The combination of file names and inode/device id are sufficient to implement persistent storage
- Drawback: constant lookups from file name to inode/device id are costly
- Idea: do expensive tree traversal once, store final inode/device number in a per-process table
  - Also keep additional information such as file offset
  - Per process table of open files
  - Use linear numbers (fd 0, 1, 2, ...), reuse when freed

File descriptor (2/4)



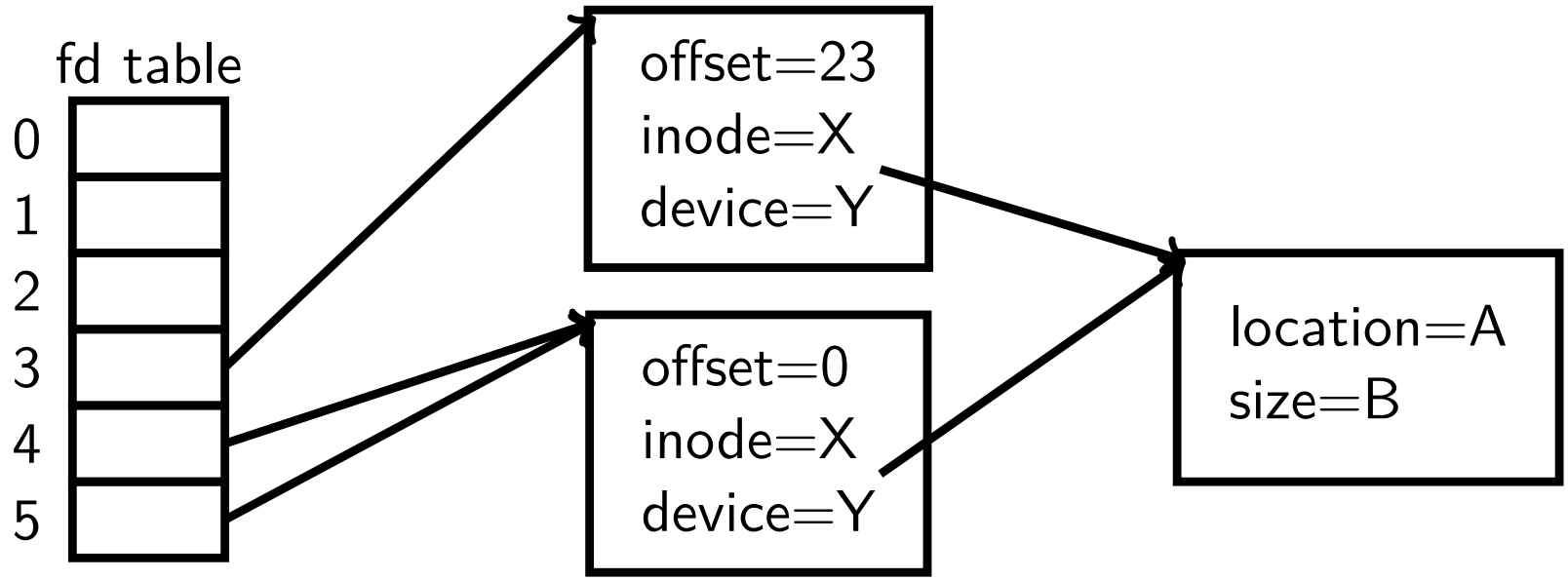

```
int fd1 = open("file.txt");  // returns 3
read(fd1, buf, 23);
```

File descriptor (3/4)




```
int fd1 = open("file.txt");  // returns 3
read(fd1, buf, 23);
int fd2 = open("file.txt");  // returns 4
```

File descriptor (4/4)




```
int fd1 = open("file.txt");   // returns 3
read(fd1, buf, 23);
int fd2 = open("file.txt");   // returns 4
int fd3 = dup(fd2);           // returns 5
```

File API

- `int open(char *path, int flag, mode_t mode)`
- `size_t read(int fd, char *buf, size_t nbyte)`
- `size_t write(int fd, char *buf, size_t nbyte)`
- `int close(int fd)`

Open translates a string name to an inode. OS allocates a file descriptor that points to that inode and returns the file descriptor table index. The path is only traversed once, the OS can cache inodes and each process keeps track of its open files.

File API: deletion

- There is no system call to delete files!
- Inodes are marked free if there are no more references to them (that's why they have a reference count)
- `unlink()` removes a file from a directory and reduces the reference count
- File descriptors are freed upon `close()` or when the process exits

Note, some programs create a temporary file, keep the file descriptor but unlink the file from the directory right after creation. This results in a private temporary file that is recycled when the process exits.

Sharing and concurrency is hard!

- Consider file permissions change *after* file is opened
- Consider a file is moved *after* it is opened
- Consider a file is deleted *after* it is opened
- Consider file owner changes *after* it is opened
- A process forks, what happens to open files (e.g., read positions)
- What happens when two processes write to the same file?
- ...

The curious case of temp files

```c
int main(int argc, char* argv[]) {
  int fd = open("test", O_CREAT | O_RDWR, 0600);
  // unlink("test"); <- what happens to fd here?
  char *data = "test";
  char rdata[64];
  write(fd, data, strlen(data));
  sleep(10);
  lseek(fd, 0, SEEK_SET);
  read(fd, &rdata, 64);
  rdata[63] = 0;
  printf("We read '%s'\n", rdata);
  close(fd);
  return 0;
}
```

More fun with filesystems: endless files



- Is it possible for `cat` to run infinitely?

More fun with filesystems: endless files



- Is it possible for `cat` to run infinitely?
- `cat /dev/zero`

Multiple file systems (1/3)

- Challenge: on a single system there are often multiple filesystems
  - Different partitions on the same disk
  - Multiple disks
  - DVD/BlueRay drive
  - USB stick
  - Network Attached Storage
  - Floppy disk
- How do you organize, manage, and display all these file systems?

Multiple file systems: Windows (2/3)

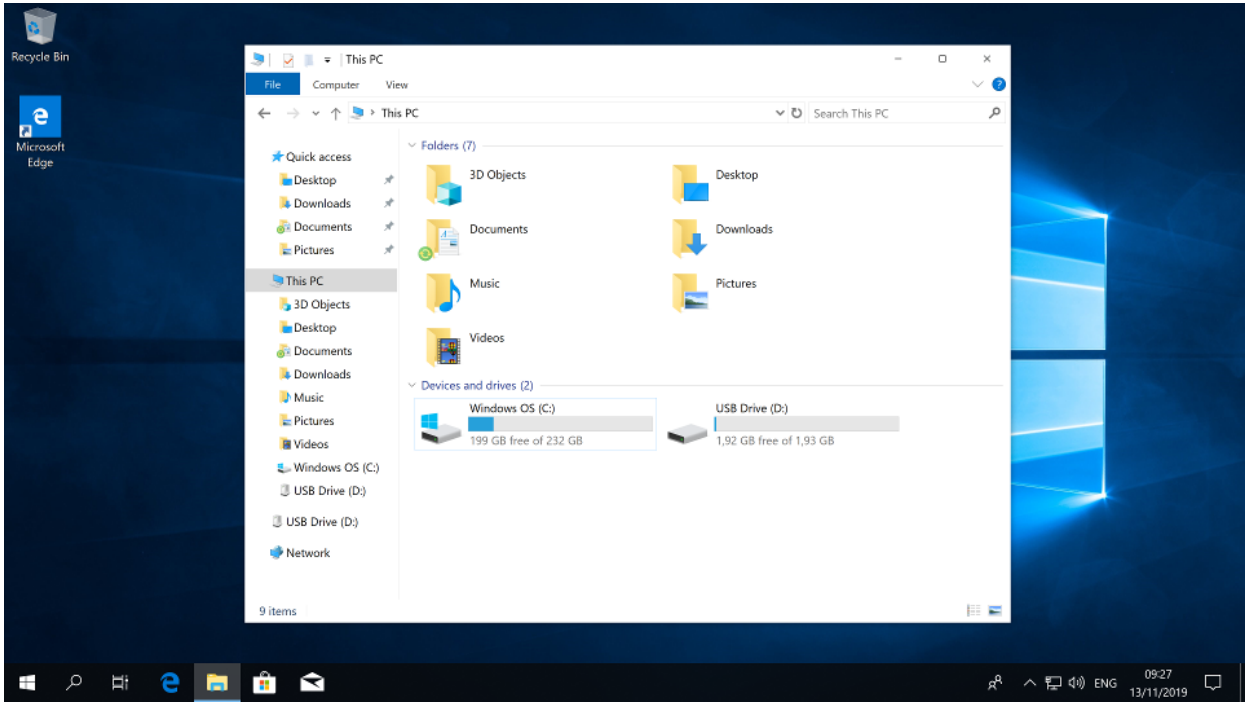
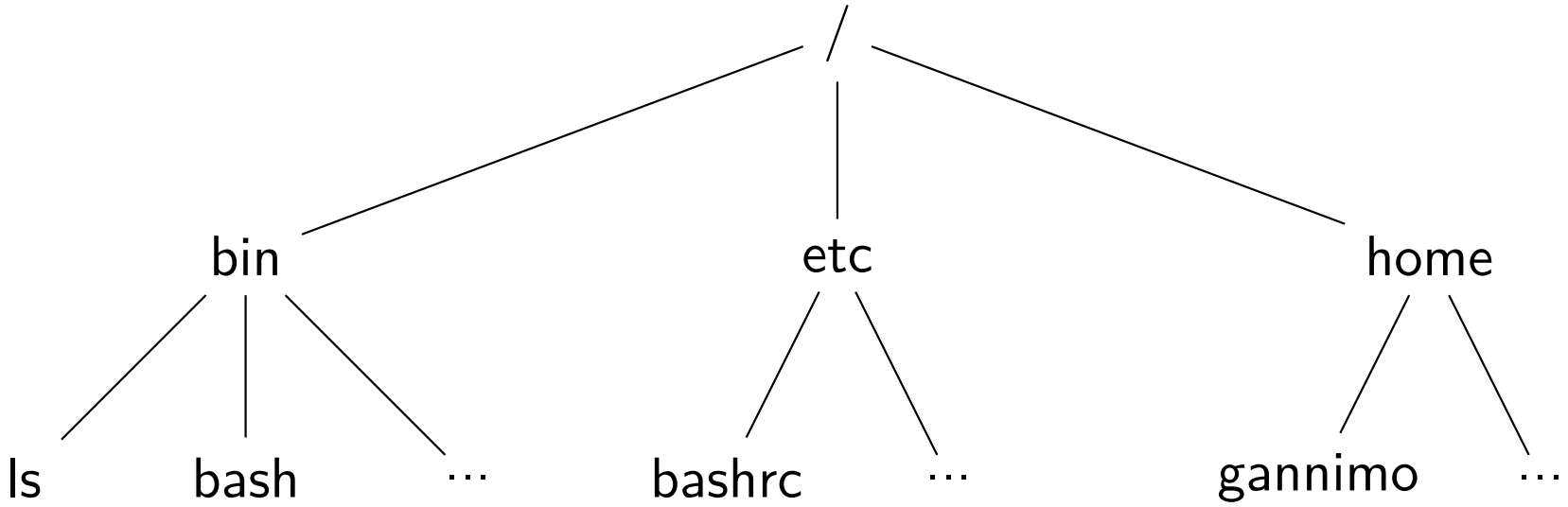


Figure 1: All file systems are mapped to a common root

- Assign a letter to each file system
  - A, B for floppy disks
  - C for main hard drive
  - ...

Multiple file systems: Unix (3/3)



- File systems can be mapped anywhere into *a single tree*
  - Any directory can be a mount point
  - Mounting a FS hides the files in the original directory
  - E.g., "home/* may be a different file system

Hard links and soft links

- Links are file pointers, i.e., they do not contain data themselves but reference another file
- Soft link: a directory entry points to a file that contains a file name, the OS resolves the file name when it is accessed
- Hard link: a directory that points to an existing file, increasing the reference counter
  - This is why the inode does not store the file name

Different names for different use cases

- inode/device id (53135/2)
  - Unique internal name
  - Records metadata about the file (size, permissions, owner)
- path (`/foo/bar/baz`)
  - Human readable name
  - Organizes files in a hierarchical layout
- file descriptor (5)
  - Process internal view
  - Avoids frequent path to inode traversal
  - Remembers offset for next read/write

Common file API

- `int open(char *path, int flag, mode_t mode)`
- `size_t read(int fd, char *buf, size_t nbyte)`
- `size_t write(int fd, char *buf, size_t nbyte)`
- `int close(int fd)`

What kind of on disk data structures do we need? How is data accessed?

Virtual File System

- File systems (EXT4, NTFS, FAT) use different data structures
- A Virtual File System (VFS) abstracts from the real filesystem
- VFS abstracts the FS as objects with specific operations
  - Superblock (mount): a life filesystem
  - File (open): a file opened by a process ("open file description")
  - Directory entry cache: speeds up path to inode translation
  - Inode (lookup): a filesystem object (e.g., file or directory)
- System call logic (open, seek, ...) maps to VFS operations
- When implementing a new FS, implement the VFS API
- System calls are now independent of FS implementation

Challenge: renaming files

- How would you implement `rename`?

Challenge: renaming files

- How would you implement `rename`?
- Renaming only changes the name of the file
- Directory contains the name of the file
- No data needs to be moved, inode remains unchanged

Challenge: renaming files

- How would you implement `rename`?
- Renaming only changes the name of the file
- Directory contains the name of the file
- No data needs to be moved, inode remains unchanged
- Note, you may need to move the data if it is on another disk/partition!

Filesystem implementation

- A filesystem is an exercise in data management
- Given: a large set (N) of blocks
- Need: data structures to encode (i) file hierarchy and (ii) per file metadata
  - Overhead (metadata size versus file data) should be low
  - Internal fragmentation should be low
  - File contents must be accessed efficiently (external fragmentation, number of metadata accesses)
  - Define operations for file API

Filesystem implementation

- A filesystem is an exercise in data management
- Given: a large set (N) of blocks
- Need: data structures to encode (i) file hierarchy and (ii) per file metadata
  - Overhead (metadata size versus file data) should be low
  - Internal fragmentation should be low
  - File contents must be accessed efficiently (external fragmentation, number of metadata accesses)
  - Define operations for file API
- Many different choices are possible!
  - Similar to virtual memory!
  - Software implementation enables experimentation with strategies

Allocating file data

- Contiguous
- Linked blocks (blocks end with a next pointer)
- File-allocation tables (table that contains block references)
- Indexed (inode contains data pointers)
- Multi-level indexed (tree of pointers)

For each approach, think about fragmentation, ability to grow/shrink files, sequential access performance, random access performance, overhead of meta data.

File allocation: contiguous

Each file is allocated contiguously



File allocation: contiguous

Each file is allocated contiguously



- Terrible external fragmentation (OS must anticipate)
- Likely unable to grow file
- Excellent read and seek performance
- Small overhead for metadata

File allocation: contiguous

Each file is allocated contiguously

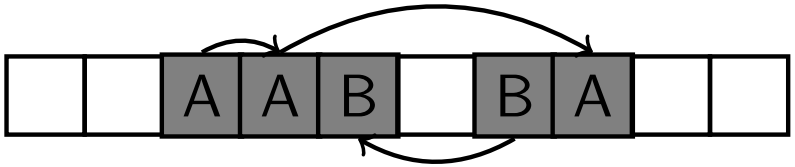


- Terrible external fragmentation (OS must anticipate)
- Likely unable to grow file
- Excellent read and seek performance
- Small overhead for metadata

- Great for read-only file systems (CD/DVD/BlueRay)

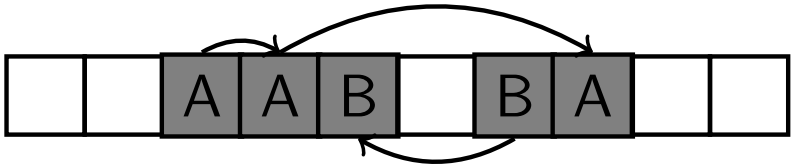
File allocation: linked blocks

Each file consists of a linked list of blocks



File allocation: linked blocks

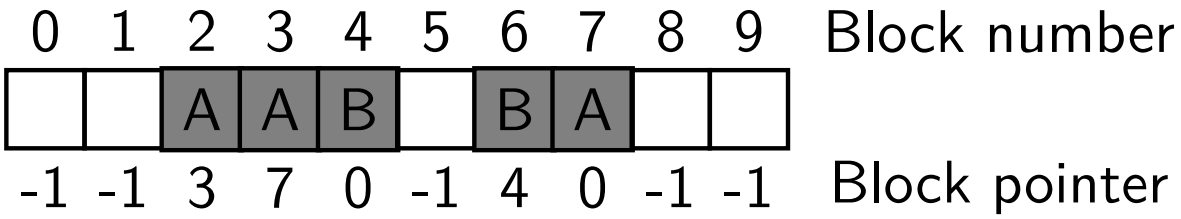
Each file consists of a linked list of blocks



- No external fragmentation
- Files can grow easily
- Reasonable read cost (depending on layout), high seek cost
- One pointer per block metadata overhead

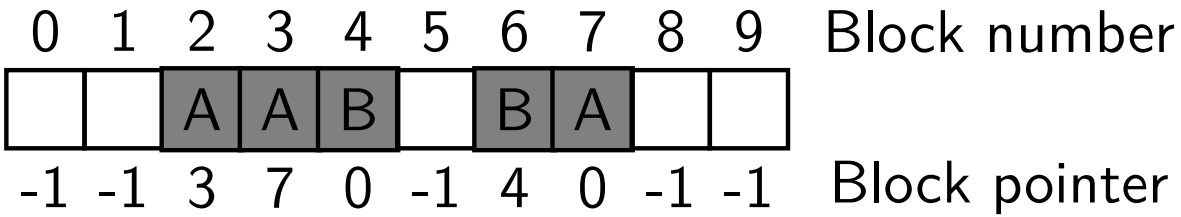
File allocation: File Allocation Table (FAT)

Idea: keep linked list information in a single table. Instead of storing the next pointer at the end of the block, store all next pointers in a central table



File allocation: File Allocation Table (FAT)

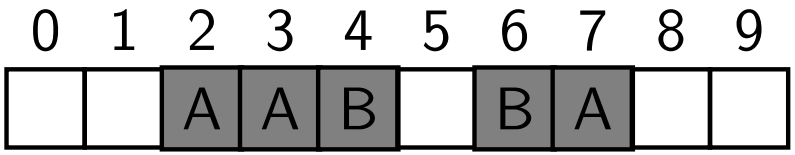
Idea: keep linked list information in a single table. Instead of storing the next pointer at the end of the block, store all next pointers in a central table



- No external fragmentation
- Files can grow easily
- Reasonable read and seek cost
- One pointer per block metadata overhead

File allocation: indexed

Idea: metadata contains an array of block pointers



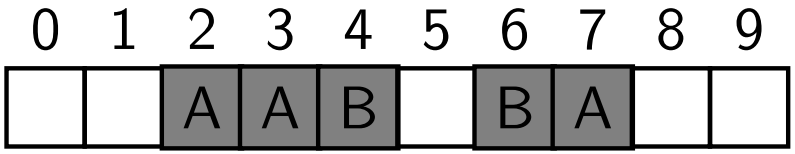
Block number

File A: 2, 3, 7, -1

File B: 6, 4, -1, -1

File allocation: indexed

Idea: metadata contains an array of block pointers



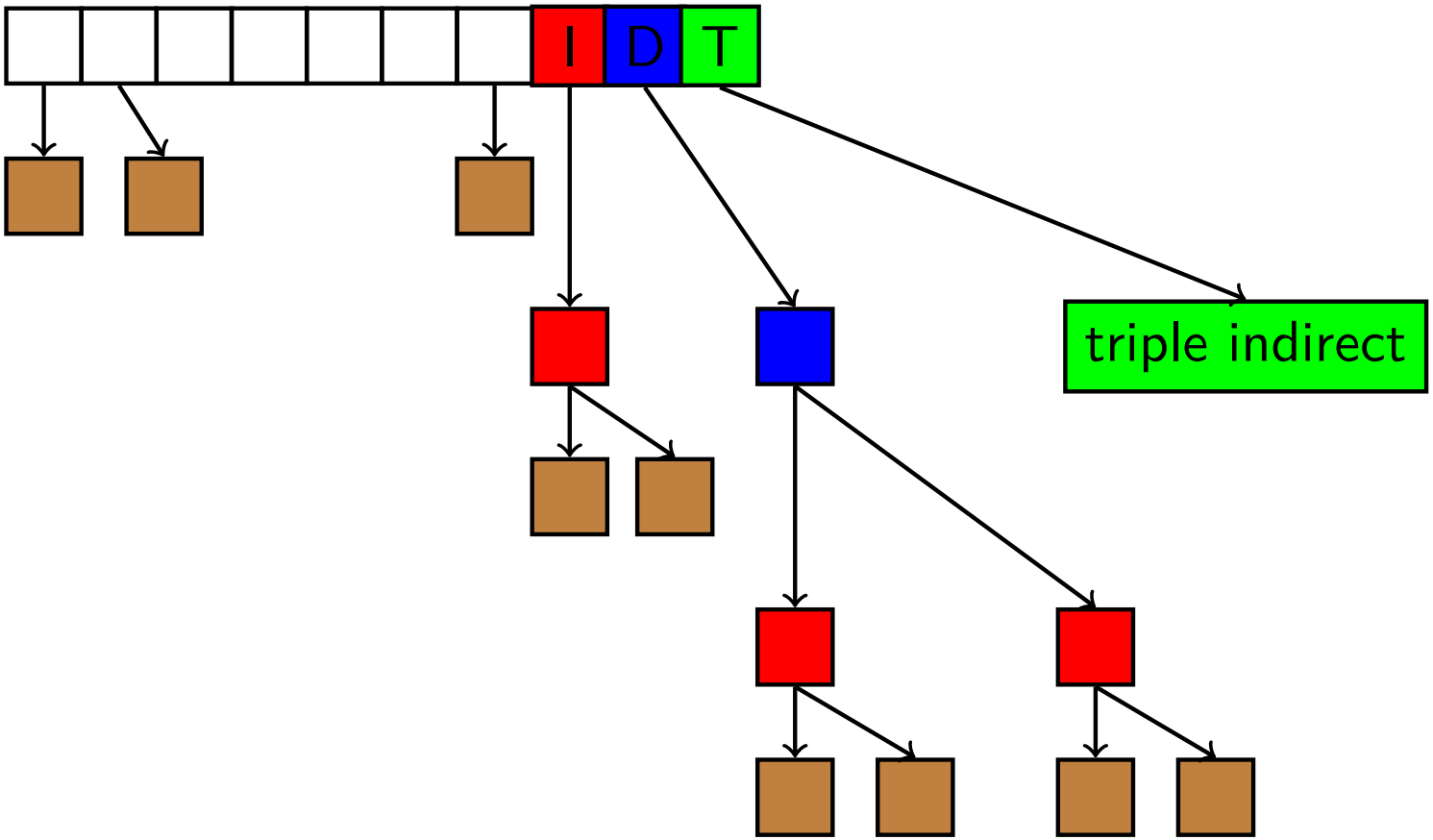
File A: 2, 3, 7, -1

File B: 6, 4, -1, -1

- No external fragmentation
- Files can grow easily up to maximum size
- Reasonable read and low seek cost
- *Large metadata overhead* (wastes space as most files are small)

File allocation: multi-level indexing (1/3)

Idea: have a mix of direct, indirect, double indirect, and triple indirect pointers



File allocation: multi-level indexing (2/3)

Idea: have a mix of direct, indirect, double indirect, and triple indirect pointers

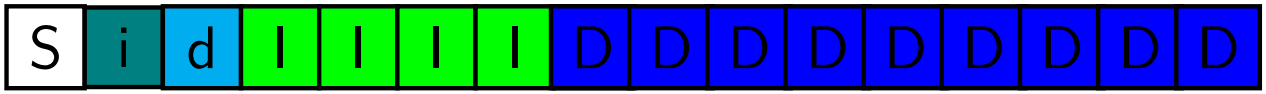
```
struct inode {
  umode_t          i_mode;
  unsigned short   i_opflags;
  kuid_t           i_uid;
  kgid_t           i_gid;
  unsigned int     i_flags;
  ...
  // direct pointers to data blocks
  struct dblock  *direct[10];
  // block of N ptrs to data blocks
  struct dblock  **dindirect;
  // block of N ptrs to each N ptrs to data blocks
  struct dblock  ***tindirect;
};
```

File allocation: multi-level indexing (3/3)

Idea: have a mix of direct, indirect, double indirect, and triple indirect pointers

- No external fragmentation
- Files can grow easily up to maximum size
- Reasonable read and low seek cost
- Low metadata overhead but needs extra reads for indirect/double indirect access

Simple FS



- Superblock (S): file system metadata
- Bitmaps (i, d): indicates free blocks
- Inodes (I): hold file/directory metadata, reference data blocks
- Data blocks (D): file contents, referenced by an inode

The inode size may be different (smaller) from the data block size.

Simple FS: superblock

- The superblock stores the characteristics of the filesystem
- What do you store in the superblock?

Simple FS: superblock

- The superblock stores the characteristics of the filesystem
- What do you store in the superblock?

- Magic number and revision level
- Mount count and maximum mount count
- Block size of the filesystem (1, 2, 4, 8, 16, 32, 64K for ext4)
- Name of the filesystem
- Number of inodes/data blocks
- Number of free inodes/data blocks
- Number of "first" inode (i.e., root directory)

Simple FS: inode

- The inode stores all file metadata
- What would you store in an inode?

Simple FS: inode

- The inode stores all file metadata
- What would you store in an inode?

- File type
- File uid, gid
- File permissions (for user, group, others)
- Size
- Access time
- Create time
- Number of links

Maximum file size

- Maximum file size is related to
  - Block size
  - Number of direct inodes
  - Number of indirect inodes
  - Number of double indirect inodes
  - Number of triple indirect inodes
- `blocksize * (direct + inodeblock + inodeblock^2 + inodeblock^3)`

Directories

- Directories are special files (`inode->type`)
- Store a set of file name to inode mappings
- Special entries `.` for current directory and `..` for parent directory

File operation: create /foo/bar

- Read root inode (locate directory data)
- Read root data (read directory)
- Read foo inode (locate directory data)
- Read foo data (read directory)
- Read/write inode bitmap (allocate inode)
- Write foo data (add file name)
- Read/write bar inode (create file)
- Write foo inode (update date, maybe allocate data block)

File operation: open /foo/bar

- Read root inode (locate directory data)
- Read root data (read directory)
- Read foo inode (locate directory data)
- Read foo data (read directory)
- Read bar inode (read file metadata)

File operation: write to /foo/bar

- First: `open("/foo/bar")`
- Read bar inode (read file metadata)
- Read/write data bitmap (allocate data blocks)
- Write bar data (write data)
- Write bar inode (update inode)

File operation: read from /foo/bar

- First: `open("/foo/bar")`
- Read bar inode (read file metadata)
- Read bar data (read data)
- Write bar inode (update time)

File operation: close /foo/bar

- No disk I/O

File operation: observations

- Path traversal and translation is costly
  - Reduce number of lookups (file descriptors!)
  - Introduce caching (dcache)
- Lookup aside, operations are cheap and local

Summary

- Filesystem API: handle interaction with the file system
- Internal and external interface
  - Internal: data structures handle large chunk of blocks
  - External: standardized interface
- Three ways to identify a file
  - File names (for humans)
  - Inodes and devices (on the disk)
  - File descriptors (for a process)
- Combine multiple file systems
  - Mount at the root (Windows)
  - Mount anywhere in the tree (Unix)
- Filesystem implementation
- Inodes for metadata
- Bitmaps for inodes/data blocks
- Superblock for global metadata